

dbt Training

● Beginner · Module 07 · 60 min · Final Beginner Module

Documentation

Not an afterthought. CI fails without it. And it feeds directly into Snowflake, Power BI, and Cortex AI.

Why Documentation Is a CI Requirement

****The real cost of missing docs**** Three months after building a model: > ****What does `fct\_prescription.dosage\_amount` represent? Is it milligrams? Is it a monetary amount? Can it be NULL?*** If the column isn't documented, the answer requires: - Reading the SQL - Tracing back through staging to Bronze - Hoping the source system has docs That's 30 minutes of investigation that a two-sentence description would have prevented.

****What CI enforces for Silver and Gold****

- ✗ No model description → PR rejected
- ✗ No grain statement → PR rejected
- ✗ Undocumented columns → PR rejected
- ✓ Bronze and Staging: optional (but encouraged)

Writing Model and Column Descriptions

```
``yaml {all|3-8|10-15|17-23|all}
```

```
version: 2
```

```
models:
```

```
- name: fct_prescription
```

```
description: >
```

Grain: one prescription event per patient per doctor per prescription_date.

Source: HubSpot deal records filtered to prescription pipeline stages.

Refreshed nightly. Excludes cancelled prescriptions.

```
columns:
  - name: prescription_key
    description: >
      Surrogate primary key. MD5 hash of (patient_key, doctor_key, prescription_date).
      Integer. Not a UUID.
    tests: [unique, not_null]

  - name: patient_key
    description: "FK to dim_patient.patient_key. Surrogate key generated by our pipeline."
    tests:
      - not_null
```

```
- relationships:
  to: ref('dim_patient')
  field: patient_key
```

```
<!--
```

Use line highlights: first show the model-level description block, then the prescription_key column, then the patient_key column with FK test.

Note that descriptions and tests live in the same schema.yml – they are not separate concerns. When you're writing tests, write descriptions. When you're writing descriptions, add tests. They belong together.

The grain statement format is mandatory: "Grain: one {entity} per {dimension1} per {dimension2}." If you can't write the grain statement, the model's design is unclear and needs a design review before documentation.

```
-->
```

```
---
```

```
# The Grain Statement
```

```
<div class="mt-4 bg-white border border-slate-200 rounded-xl p-6">
  <div class="font-semibold text-slate-700 mb-3">Mandatory format for Silver and Gold models</div>
```

Grain: one {entity} per {dimension1} per {dimension2} per {time dimension}.

```
</div>
```

```
<div class="mt-4 space-y-3">
```

```
<div class="bg-emerald-50 border border-emerald-200 rounded-lg p-4 text-sm">
  <div class="font-mono text-emerald-600 text-xs mb-1">✅ Good grain statement</div>
  <div>"Grain: one prescription event per patient per doctor per prescription_date."</div>
</div>
```

```
<div class="bg-red-50 border border-red-200 rounded-lg p-4 text-sm">
  <div class="font-mono text-red-500 text-xs mb-1">❌ Too vague – CI will reject</div>
  <div>"Grain: prescriptions." – What does one row represent exactly?</div>
</div>
```

```
<div class="bg-amber-50 border border-amber-200 rounded-lg p-4 text-sm">
  <div class="font-mono text-amber-600 text-xs mb-1">⚠️ If you can't write the grain, the design is
unclear</div>
  <div>Stop. Review the model structure before writing documentation. A grain statement that's hard to write
is a design smell.</div>
</div>
```

```
</div>
```

```
<!--
```

The grain statement is the most important sentence in any data model. It defines the semantic meaning of a row.

Without it, someone querying the model can't know if they should aggregate, deduplicate, or filter. With it, they can write correct SQL immediately.

The "if you can't write it, the design is unclear" point is important. A grain statement that takes three attempts is telling you something about the model – it might have mixed granularity, implicit filters, or SCD2 complications that should be made explicit.

Ask: "Write a grain statement for fct_prescription." Someone should be able to answer: "Grain: one prescription event per patient per doctor per prescription_date."

```
-->
```

```
---
```

```
# Doc Blocks – For Longer Descriptions
```

```
<div class="mt-4 grid grid-cols-2 gap-4">
```

```
<div>
```

```
***`models/silver/_silver_docs.md`**
```

```
```markdown
```

```
{% docs fct_prescription %}
```

```
One prescription per patient per doctor per date.
```

```
Source: HubSpot deals in prescription pipeline stages.
```

```
Excludes cancelled prescriptions (status != closed_won).
```

```
Refreshed nightly after the Bronze load completes.
```

```
{% enddocs %}
```

```
***`schema.yml`**
```

```
models:
```

```
- name: fct_prescription
```

```
 description: '{{ doc("fct_prescription") }}'
```

```
 columns:
```

```
 - name: prescription_key
```

```
 description: "Surrogate PK. MD5 hash."
```

Short inline descriptions are the standard. Use doc blocks only when a model's business context genuinely needs more than one or two lines — typically complex Silver facts or models with non-obvious exclusion logic.

## **persist\_docs** — Docs Flow into Snowflake

**\*\*Already configured in `dbt\_project.yml`\*\***

```
models:
 analytics:
 silver:
 +persist_docs:
 relation: true # model description → table comment
 columns: true # column descriptions → column comments
```

**\*\*What dbt runs after every Silver/Gold build:\*\***

```
COMMENT ON TABLE SILVER.PUBLIC.fct_prescription
IS 'Grain: one prescription event per...';

COMMENT ON COLUMN SILVER.PUBLIC.fct_prescription.prescription_key
IS 'Surrogate primary key. MD5 hash of...';
```

Snowsight

Hover over a column in the table browser → your dbt description appears as a tooltip

Power BI

Column descriptions appear in the field list — visible to report developers without opening dbt

Snowflake Cortex AI

Column comments are used as context for natural language queries — better descriptions = better AI answers

## **dbt docs generate and dbt docs serve**

```
Step 1: Build the docs artifact
dbt docs generate

Step 2: Serve locally
dbt docs serve

Opens http://localhost:8080
```

→ Model list with descriptions

→ DAG view — full lineage graph

→ Column panel with types, descriptions, tests

→ Source freshness status

The DAG is your best debugging tool

"If I change dim\_patient, what downstream models are affected?" — the DAG answers this in 5 seconds.

How to reach the DAG

Click any model → bottom panel → "See lineage" → full upstream/downstream graph

## Exercise: Document `dim_pipeline` (20 min)

The model exists with no documentation. Write a complete `schema.yml` entry.

Column	Type	Description
<code>`pipeline_key`</code>	Integer	Surrogate PK
<code>`hubspot_pipeline_id`</code>	String	Business key from HubSpot
<code>`pipeline_name`</code>	String	Human-readable name
<code>`is_active`</code>	Boolean	Whether pipeline is currently in use
<code>`dbt_valid_from`</code>	Timestamp	SCD2 validity start
<code>`dbt_valid_to`</code>	Timestamp	SCD2 validity end (NULL = current record)
<code>`is_current`</code>	Boolean	True if active version of this record

**Hint on grain:** This is SCD2 — one row per pipeline per validity period. The grain must reflect that.

layout: center

 Beginner Tier Complete

### You can now:

- ✓ Explain what dbt is and where it fits in the data stack
- ✓ Navigate the project and understand `dbt_project.yml` and `profiles.yml`
- ✓ Read and write Jinja: `ref()`, `source()`, `config()`, `{% if %}`
- ✓ Choose the correct materialization for any use case
- ✓ Declare sources and write staging models that reference them correctly
- ✓ Write complete test suites using all four generic tests
- ✓ Write model and column documentation that passes CI
- ✓ Use `dbt build`, `dbt docs serve`, and `dbt source freshness`

Intermediate tier starts with Module 08 — Advanced Incremental Patterns